

Optimización
Multiplicación de matrices $n \times n$

Issabella del Pilar Ortiz

Santiago Talero Parra

Juan Sebastian Manrique Moreno

Algoritmos utilizados

Los algoritmos utilizados para la optimización fueron los siguientes:

1. Multiplicación normal optimizada por ciclo `for`.
2. Multiplicación fila a fila con $\mathbb{A} \cdot \mathbb{B}^T$.
3. Optimización por compilación con las banderillas `-O0`, `-O2`, `-O3`.

Investigación: *uso de la memoria caché.*

Preámbulo del código (antes del main)

Inicialmente se declaran las librerías a utilizar en el código, junto a algunas funciones que sirven como utilidades dentro del código, tales como:



```
#include <iostream>
#include <vector>
#include <chrono>
#include <functional>
#include <cstdlib>

using namespace std;
using namespace chrono;

using Matrix = vector<vector<int>>>;
```

Preámbulo del código (antes del main)

Inicialmente se declaran las librerías a utilizar en el código, junto a algunas funciones que sirven como utilidades dentro del código, tales como:

- `inicializarMatriz(int n)`: su función es crear las matrices A y B con números aleatorios entre 0 y 9.
- `trasponeMatriz(const Matrix &mat)`: su función es transponer la matriz dada.
- `void medirTiempo(const string &nombre, function<void()> funcion)`: su función es medir el tiempo que demora en ejecutarse ciertos bloques de código.
- `bool comparaMatrices(const Matrix &A, const Matrix &B)`: su función es comparar dos matrices e indicar si son iguales o no.

```
// ----- Utilidades -----

Matrix inicializarMatriz(int n)
{
    Matrix mat(n, vector<int>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            mat[i][j] = rand() % 10;
    return mat;
}

Matrix transponeMatriz(const Matrix &mat)
{
    int n = mat.size();
    Matrix transpuesta(n, vector<int>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            transpuesta[j][i] = mat[i][j];
    return transpuesta;
}

void medirTiempo(const string &nombre, function<void()> funcion)
{
    auto inicio = high_resolution_clock::now();
    funcion();
    auto fin = high_resolution_clock::now();
    duration<double> duracion = fin - inicio;
    cout << "\u23F1 " << nombre << ": " << duracion.count() << " segundos\n";
}

bool compararMatrices(const Matrix &A, const Matrix &B)
{
    int n = A.size();
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (A[i][j] != B[i][j])
                return false;
    return true;
}
```

Inicialización (main)

Luego, se inicia con las matrices que contendrán las respuestas (C1, C2, C3 y C4) junto a las matrices del producto A, B y su traspuesto BT. A su vez, la dimensión de la matriz cuadrada n.

Se crean las matrices A y B.

```
// ----- Main -----  
  
int main()  
{  
    const int n = 1024; // Tamaño de la matriz  
    Matrix A, B, BT;  
    Matrix C1(n, vector<int>(n, 0)); // A x B (fila x columna)  
    Matrix C2(n, vector<int>(n, 0)); // A x B optimizado  
    Matrix C3(n, vector<int>(n, 0)); // A x B^t (fila x fila)  
    Matrix C4(n, vector<int>(n, 0)); // A x B^t optimizado  
  
    // 1. Inicialización  
    medirTiempo("Generación de matrices", [&]()  
    {  
        A = inicializarMatriz(n);  
        B = inicializarMatriz(n);  
    });  
}
```

Multiplicación tradicional

El primer algoritmo utilizado fue la multiplicación tradicional, esta se compone de la forma fila – columna, esto es, se crean 3 ciclos **for**, el primero accede a la fila de la matriz A, el segundo accede a la columna de la fila A y la columna de la fila B, el tercero recorre las filas de B.

Este algoritmo tiene una cantidad de cache miss aproximado a la cantidad de filas y columnas que tiene B, lo que lo hace poco eficiente.

```
// 2. Multiplicación A * B (fila * columna)
medirTiempo("Multiplicación A * B (fila * columna)", [&]()
{
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) {
            int suma = 0;
            for (int k = 0; k < n; ++k)
                suma += A[i][k] * B[k][j];
            C1[i][j] = suma;
        }
});
```

Multiplicación optimizada (i-k-j)

El segundo algoritmo utilizado fue la multiplicación optimizada mediante el orden del ciclo `for`. Este algoritmo lo que hace es ingresar a la fila `i` y luego recorrer las columnas `k`, guarda el elemento de matriz `A[i][k]` para que estando en `k`, tome este como fila y la recorra con `j`.

Esta organización permite ingresar a los espacios de memorias como filas lo cual es óptimo debido a como `C++` guarda las matrices.

```
// 3. Multiplicación A * B optimizada (i-k-j)
medirTiempo("Multiplicación A * B optimizada (i-k-j)", [&]()
{
    for (int i = 0; i < n; ++i)
        for (int k = 0; k < n; ++k) {
            int a_ik = A[i][k];
            for (int j = 0; j < n; ++j)
                C2[i][j] += a_ik * B[k][j];
        }
});
```

Multiplicación directa por traspuesta

El tercer algoritmo utilizado se basa en trasponer la matriz B y realizar una multiplicación tradicional pero ahora término a término, fila a fila. Al igual que el algoritmo anterior, en este caso solamente se está accediendo a filas desde la RAM para pasar al caché, lo que implica que es un método muy óptimo en comparación.

```
// 4. Transposición de B
medirTiempo("Transposición de B", [&]()
{ BT = transponerMatriz(B); });

// 5. Multiplicación A × Bt (fila × fila)
medirTiempo("Multiplicación A × Bt (fila × fila)", [&]()
{
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) {
            int suma = 0;
            for (int k = 0; k < n; ++k)
                suma += A[i][k] * BT[j][k];
            C3[i][j] = suma;
        } });
```


Multiplicación directa por traspuesta ¿optimizada?

El cuarto algoritmo utilizado en realidad es la combinación de los dos últimos bajo la siguiente premisa: *si reorganizar los ciclos for entrega una mayor optimización y multiplicar por la traspuesta también, ¿por qué no usarlos ambos?*

Esta pregunta se implementó directamente en este último algoritmo arrojando resultados muy particulares los cuales son a principio contraintuitivos, pero luego de un análisis tienen mucho sentido.

```
// 6. Multiplicación A × Bt optimizada (i-k-j)
medirTiempo("Multiplicación A × Bt optimizada (i-k-j)", [&]{
    {
        for (int i = 0; i < n; ++i)
            for (int k = 0; k < n; ++k) {
                int a_ik = A[i][k];
                for (int j = 0; j < n; ++j)
                    C4[i][j] += a_ik * BT[j][k];
            }
    });

// 7. Comparación de resultados
cout << "\n\U0001F4CA Comparación de resultados:\n";
cout << "- C1 vs C2: " << (compararMatrices(C1, C2) ? "✅ Iguales" : "❌ Diferentes") <<
endl; cout << "- C1 vs C3: " << (compararMatrices(C1, C3) ? "✅ Iguales" : "❌ Diferentes") <<
endl; cout << "- C1 vs C4: " << (compararMatrices(C1, C4) ? "✅ Iguales" : "❌ Diferentes") <<
endl;
return 0;
}
```

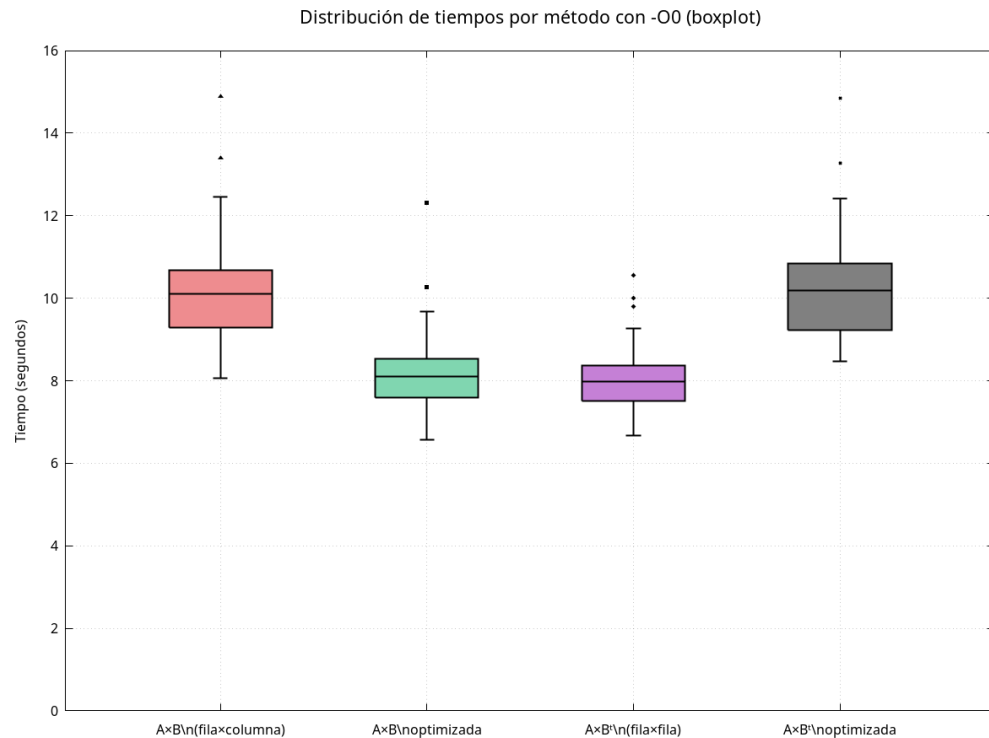
Resultados de una muestra de datos

- Optimización con 00
- Optimización con 02
- Optimización con 03

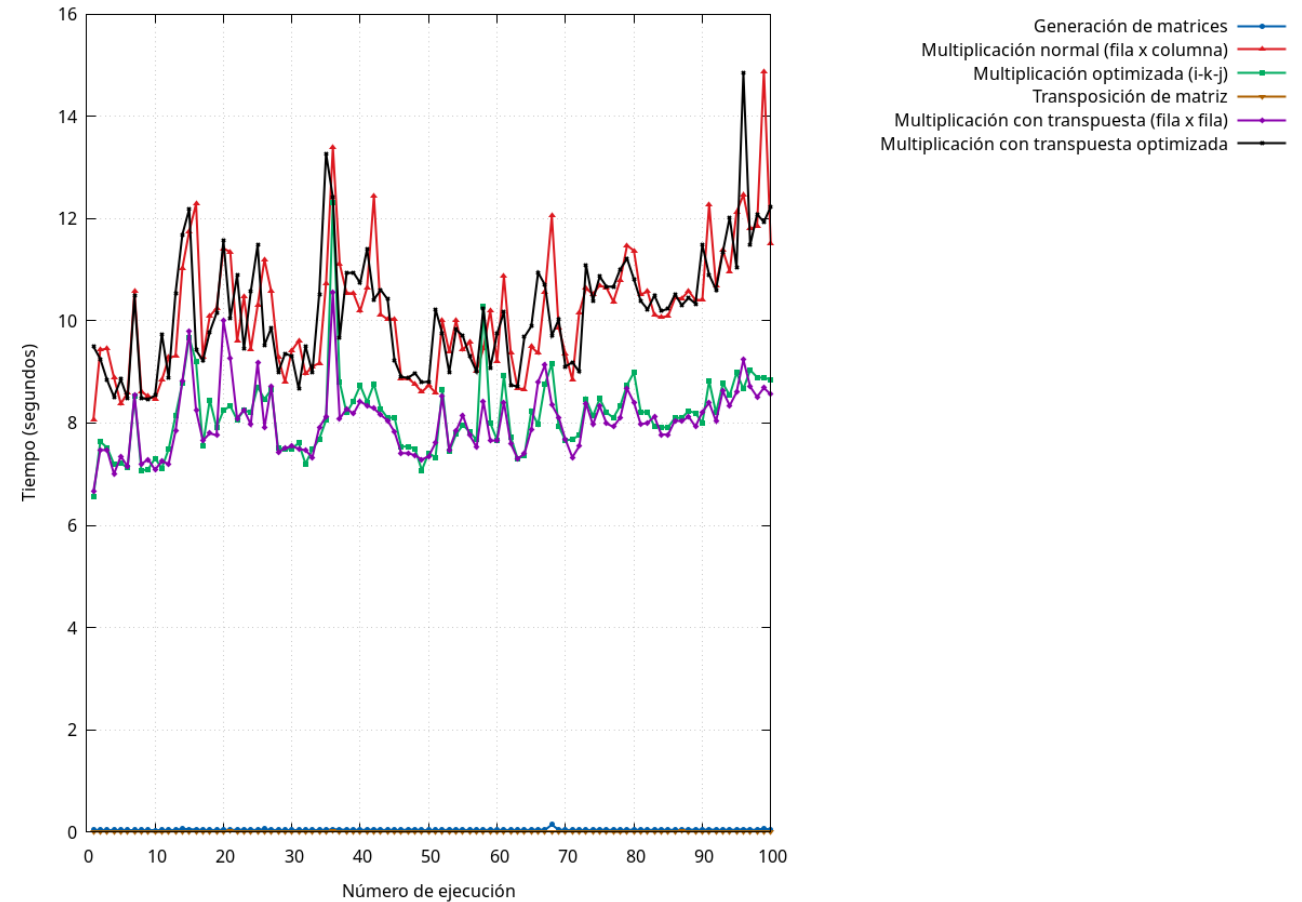
Optimización: Oo

Resultados Tiempos Promedios:

- *Multiplicación Normal: 10157.26 ms*
- *Multiplicación Optimizada: 8133.74 ms*
- *Multiplicación Transpuesta: 8025.79 ms*
- *Multiplicación Transpuesta Optimizada: 10158.78 ms*



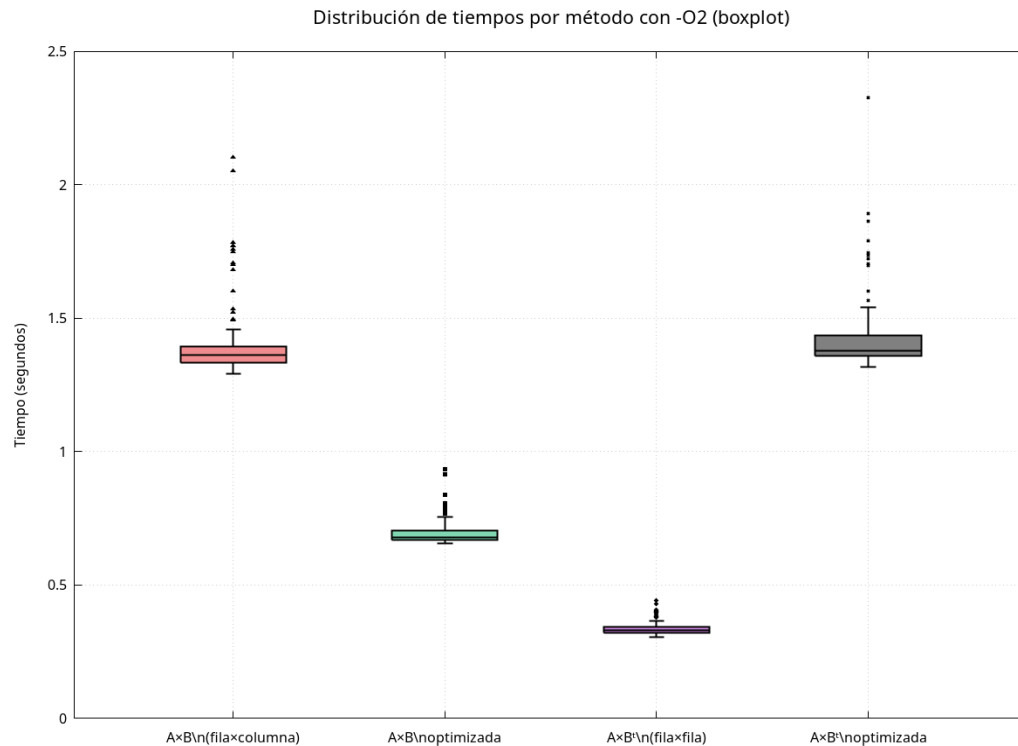
Tiempos de ejecución para operaciones matriciales con -O0 (100 ejecuciones)



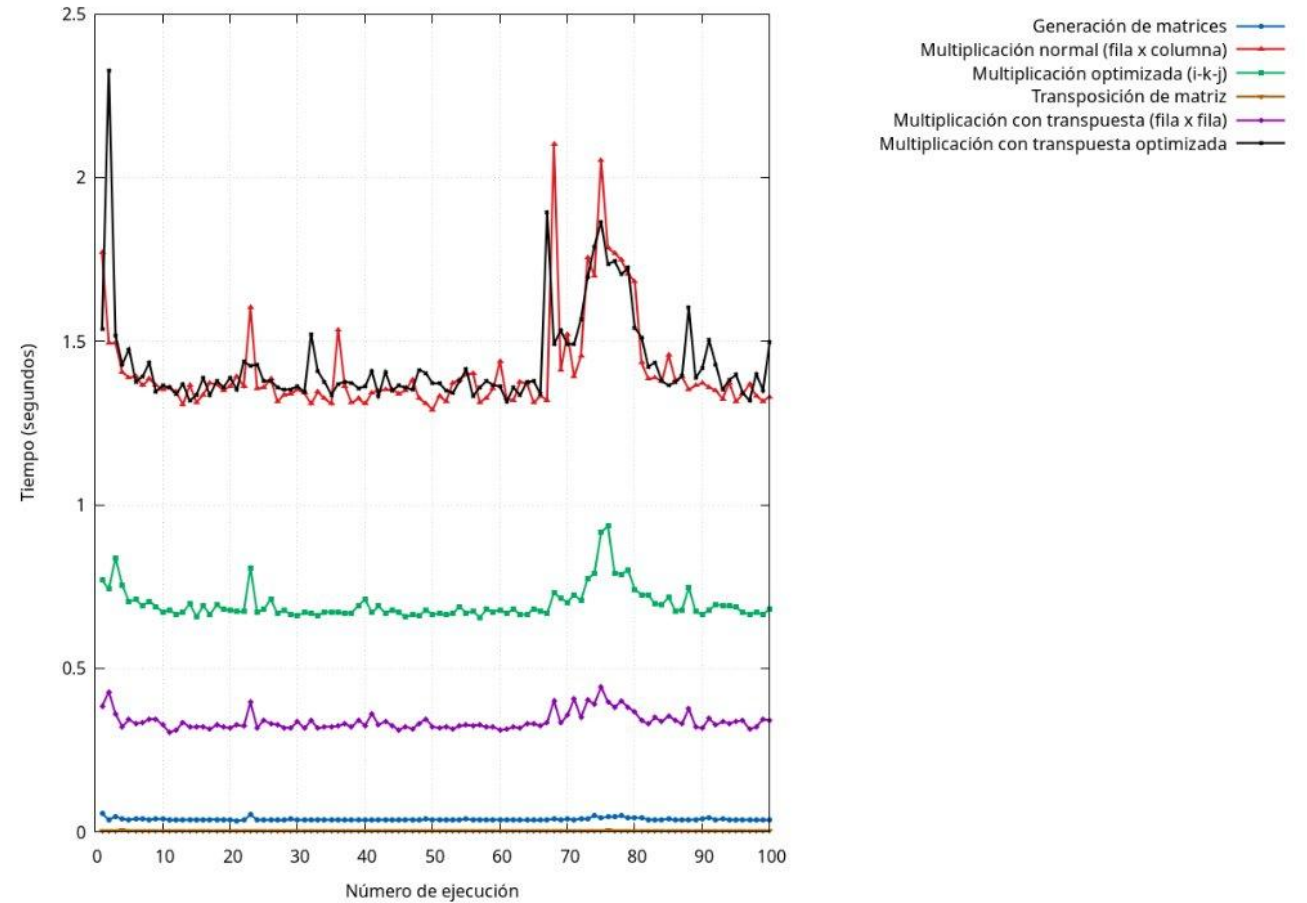
Optimización: O2

Resultados Tiempos Promedios:

- *Multiplicación Normal: 1409.52 ms*
- *Multiplicación Optimizada: 697.60 ms*
- *Multiplicación Transpuesta: 338.81 ms*
- *Multiplicación Transpuesta Optimizada: 1434.63ms*



Tiempos de ejecución para operaciones matriciales con -O2 (100 ejecuciones)

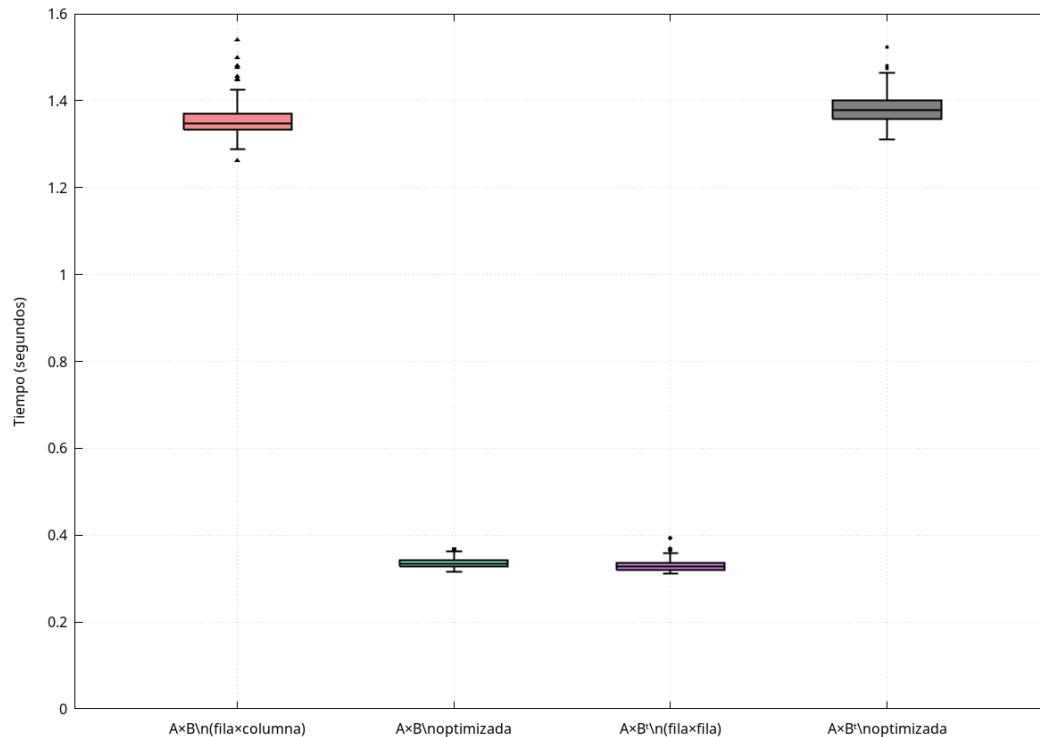


Optimización: O₃

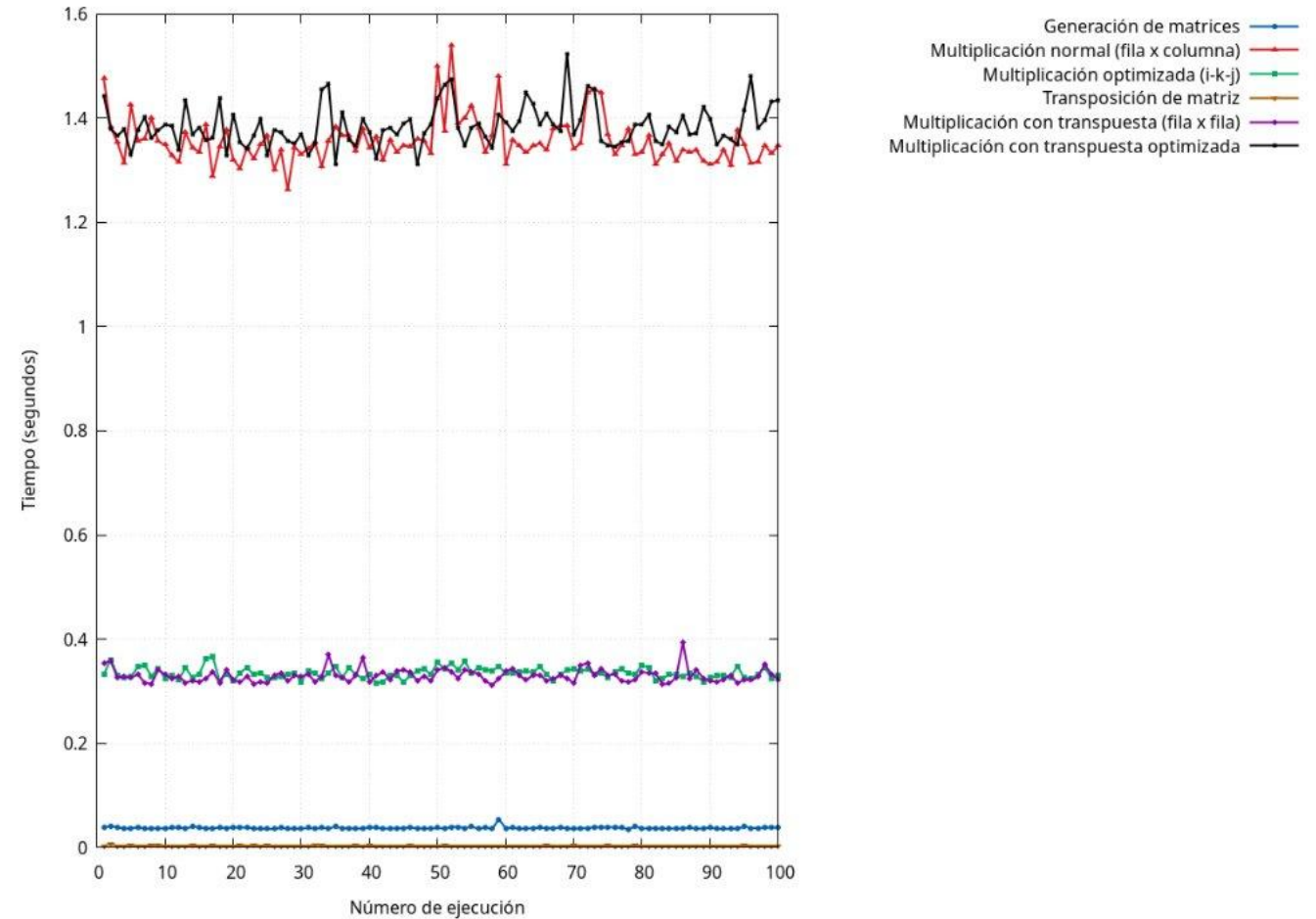
Resultados Tiempos Promedios:

- *Multiplicación Normal: 1356.50 ms*
- *Multiplicación Optimizada: 334.76 ms*
- *Multiplicación Transpuesta: 329.76 ms*
- *Multiplicación Transpuesta Optimizada: 1384.62 ms*

Distribución de tiempos por método con -O3 (boxplot)



Tiempos de ejecución para operaciones matriciales con -O3 (100 ejecuciones)



Modelo de caché ideal

- Se usa un modelo donde la memoria está dividida en **líneas de caché** de tamaño donde la caché tiene una capacidad de Z elementos.
- Cada acceso a memoria puede generar un **cache hit** (cuando el dato ya está en la caché) o una **cache miss** (cuando hay que traerlo desde la memoria principal).
- En lugar de multiplicar matrices completas directamente, se dividen en **submatrices** de tamaño $s \times s$.

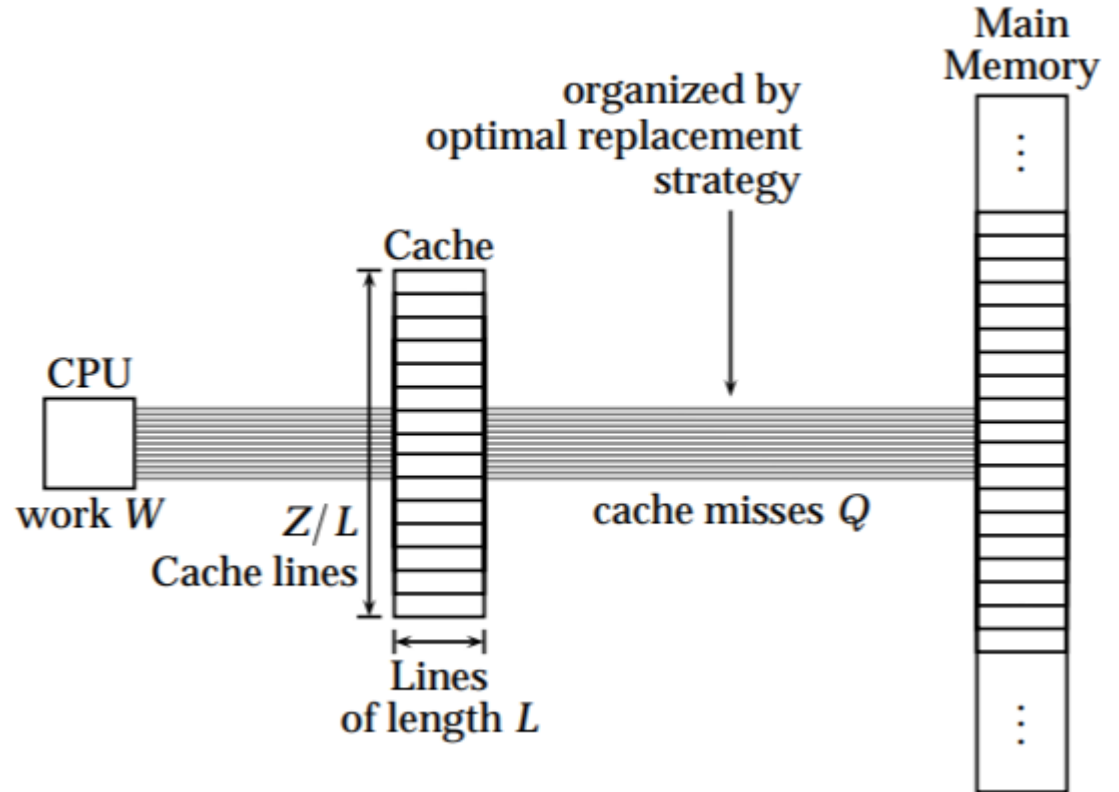


Figura xx: Modelo de cache ideal. Tomado de [1]

Modelo de caché ideal

- Esto permite que los bloques se mantengan más tiempo en la caché, reduciendo el número de accesos a memoria principal.
- Cada bloque tiene s^2 elementos, por lo que para 3 matrices $3s^2 \leq Z$

$$s = \Theta\sqrt{Z}$$

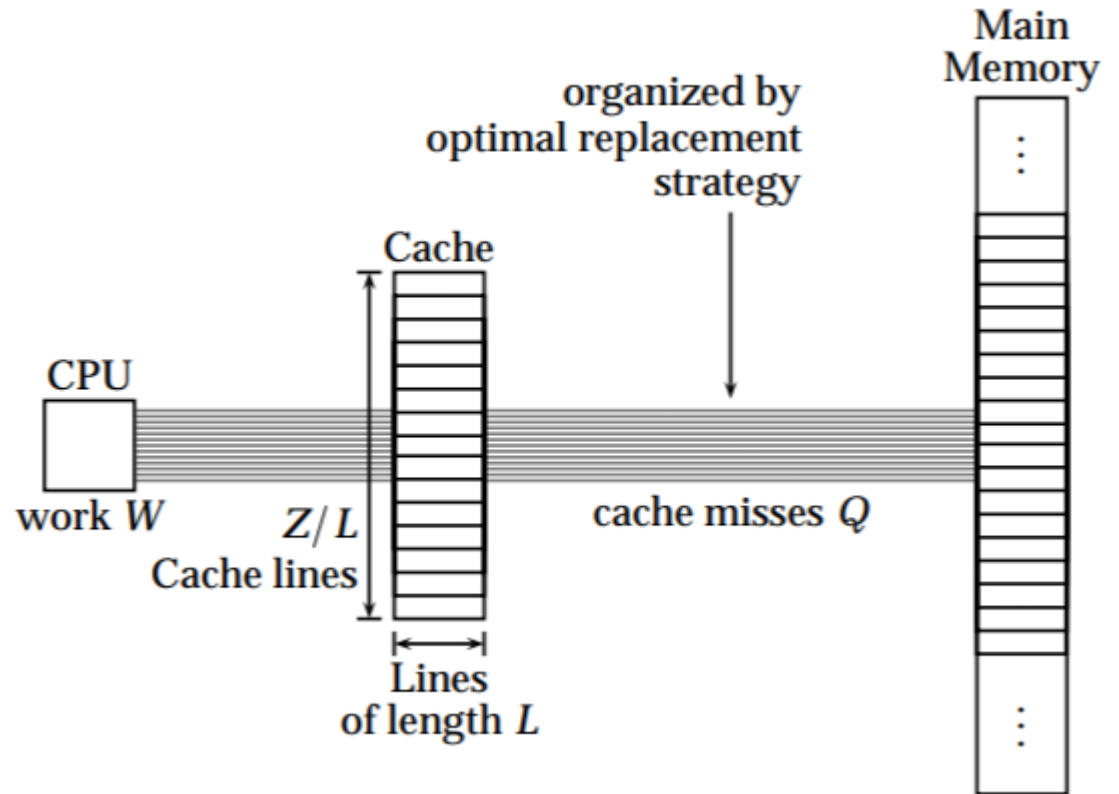


Figura xx: Modelo de cache ideal. Tomado de [1]

Almacenamiento de $s \times s$

- Una fila de la submatriz ocupa aproximadamente s/L líneas de cache.

$$\Theta\left(s + s^2/L\right)$$

- En donde s proviene del hecho de que incluso si s es menor que L , se necesita al menos una línea por fila.

- s^2/L proviene del caso en el que una fila completa no cabe en una sola línea, por lo que hay un número adicional de líneas de caché necesarias.

BLOCK-MULT algoritmo

- Con el modelo para almacenar información de cada matriz obtenemos $(n/s)^3$, por ende:

$$\Theta\left((n/s)^3(s + s^2/L)\right)$$

- Sustituyendo en $s = \Theta\sqrt{Z}$

$$\Theta\left(n + n^2/L + n^3/L\sqrt{Z}\right)$$

- $\Theta(n)$: costo mínimo de acceder a cada elemento al menos una vez.
- $\Theta(n^2/L)$: Acceso a grupos de L elementos a la vez.
- $\Theta(n^3/L\sqrt{Z})$: los cache misses causados por la transferencia de bloques de datos cuando la matriz no cabe completamente en la caché.

¡Gracias!